

Assignment-3

1. Create a Flask application with an `/api` route. When this route is accessed, it should return a JSON list. The data should be stored in a backend file, read from it, and sent as a response.

```
app_new.py > ...
1  from flask import Flask, jsonify
2  import json
3  import os
4
5  app = Flask(__name__)
6
7  # Path to the data file
8  DATA_FILE = os.path.join(os.path.dirname(__file__), 'data.json')
9
10 @app.route('/')
11 def home():
12     return jsonify({'message': 'Welcome to Flask API'})
13
14 @app.route('/api')
15 def api():
16     """Return JSON data from data.json file"""
17     try:
18         with open(DATA_FILE, 'r') as file:
19             data = json.load(file)
20             return jsonify(data)
21     except FileNotFoundError:
22         return jsonify({'error': 'Data file not found'}), 404
23     except json.JSONDecodeError:
24         return jsonify({'error': 'Invalid JSON in data file'}), 500
25
26 if __name__ == '__main__':
27     app.run(debug=True)
28
```

Objective

This program creates a simple REST API using the Flask framework. It provides two endpoints:

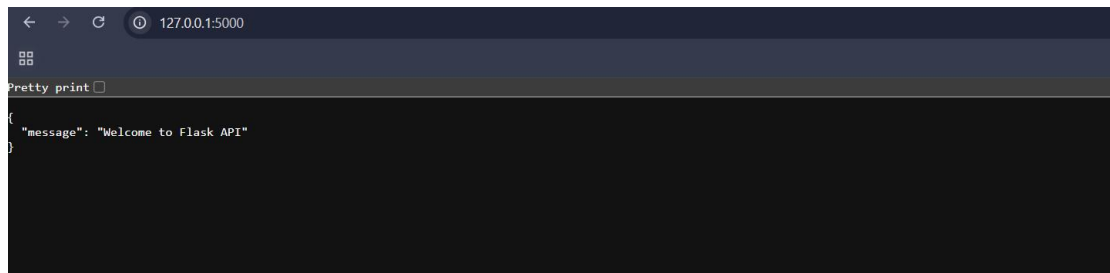
1. **Home Route (/)** – Displays a welcome message.
2. **API Route (/api)** – Reads data from a JSON file and returns it as a JSON response.

Data.json

```
data.json > {} products > {} 2
1 {
2   "users": [
3     {
4       "id": 1,
5       "name": "John Doe",
6       "email": "john@example.com",
7       "status": "active"
8     },
9     {
10      "id": 2,
11      "name": "Jane Smith",
12      "email": "jane@example.com",
13      "status": "active"
14    },
15    {
16      "id": 3,
17      "name": "Bob Johnson",
18      "email": "bob@example.com",
19      "status": "inactive"
20    },
21    {
22      "id": 4,
23      "name": "Alice Brown",
24      "email": "alice@example.com",
25      "status": "active"
26    }
27  ],
28  "products": [
29    {
30      "id": 101,
31      "name": "Laptop",
32      "price": 999.99,
33      "category": "Electronics"
```

```
27 ],
28 "products": [
29 {
30   "id": 101,
31   "name": "Laptop",
32   "price": 999.99,
33   "category": "Electronics"
34 },
35 {
36   "id": 102,
37   "name": "Mouse",
38   "price": 29.99,
39   "category": "Accessories"
40 },
41 {
42   "id": 103,
43   "name": "Keyboard",
44   "price": 79.99,
45   "category": "Accessories"
46 }
47 ]
48 }
49 }
```

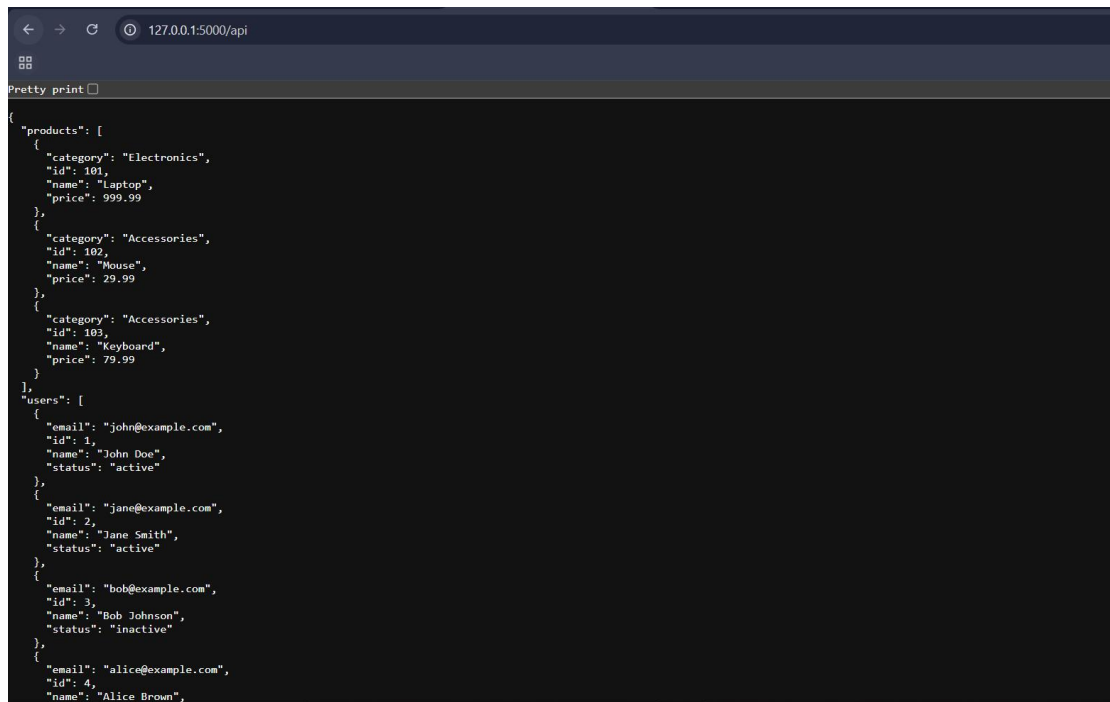
Home Route (http://localhost:5000/)



A screenshot of a web browser window with the address bar showing '127.0.0.1:5000'. The page content is a JSON object displayed in a 'Pretty print' format. The JSON object has a single key 'message' with the value 'Welcome to Flask API'.

```
{
  "message": "Welcome to Flask API"
}
```

API Route (http://localhost:5000/api)



A screenshot of a web browser window with the address bar showing '127.0.0.1:5000/api'. The page content is a JSON object displayed in a 'Pretty print' format. The JSON object has two main keys: 'products' and 'users'. 'products' is an array of three objects, and 'users' is an array of four objects.

```
{
  "products": [
    {
      "category": "Electronics",
      "id": 101,
      "name": "Laptop",
      "price": 999.99
    },
    {
      "category": "Accessories",
      "id": 102,
      "name": "Mouse",
      "price": 29.99
    },
    {
      "category": "Accessories",
      "id": 103,
      "name": "Keyboard",
      "price": 79.99
    }
  ],
  "users": [
    {
      "email": "john@example.com",
      "id": 1,
      "name": "John Doe",
      "status": "active"
    },
    {
      "email": "jane@example.com",
      "id": 2,
      "name": "Jane Smith",
      "status": "active"
    },
    {
      "email": "bob@example.com",
      "id": 3,
      "name": "Bob Johnson",
      "status": "inactive"
    },
    {
      "email": "alice@example.com",
      "id": 4,
      "name": "Alice Brown",
      "status": "inactive"
    }
  ]
}
```

2. Create a form on the frontend that, when submitted, inserts data into MongoDB Atlas. Upon successful submission, the user should be redirected to another page displaying the message **"Data submitted successfully"**. If there's an error during submission, display the error on the same page without redirection.

app_mongo.py

```
app_mongo.py > form
1 from flask import Flask, render_template, request, redirect, url_for, jsonify
2 from pymongo import MongoClient
3 from pymongo.errors import PyMongoError
4 import os
5 from dotenv import load_dotenv
6
7 load_dotenv()
8
9 app = Flask(__name__)
10
11 # MongoDB Atlas connection string
12 # Replace with your actual MongoDB Atlas URI
13 MONGO_URI = os.environ.get('MONGO_URI', 'mongodb+srv://username:password@cluster.mongodb.net/database?retryw
14
15 try:
16     client = MongoClient(MONGO_URI)
17     db = client['flask_app']
18     collection = db['submissions']
19     # Test connection
20     client.admin.command('ping')
21     print("✓ Connected to MongoDB Atlas")
22 except PyMongoError as e:
23     print(f"✗ MongoDB connection error: {e}")
24
25 @app.route('/')
26 def form():
27     """Display the data submission form"""
28     return render_template('form.html')
29
30 @app.route('/submit', methods=['POST'])
31 def submit():
32     """Handle form submission and insert data into MongoDB"""
33     try:
34         # Get form data
35         data = {
36             'name': request.form.get('name'),
37             'email': request.form.get('email'),
38             'message': request.form.get('message'),
39             'phone': request.form.get('phone')
40         }
41
42         # Validate required fields
43         if not all([data['name'], data['email'], data['message']]):
44             return render_template('form.html', error='All fields are required!')
45
46         # Insert into MongoDB
47         result = collection.insert_one(data)
48
49         # Redirect to success page
50         return redirect(url_for('success'))
51
52     except PyMongoError as e:
53         return render_template('form.html', error=f'Database error: {str(e)}')
54     except Exception as e:
55         return render_template('form.html', error=f'An error occurred: {str(e)}')
56
```

```

56
57 @app.route('/success')
58 def success():
59     """Display success message after submission"""
60     return render_template('success.html')
61
62 if __name__ == '__main__':
63     app.run(debug=True)
64

```

Objective

This program is a Flask web application that allows users to submit data through an HTML form and stores the submitted information in a MongoDB Atlas database. It also validates user input and displays a success page after successful submission.

Success.html

```

templates > <> success.html > ...
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>Submission Successful</title>
7      <style>
8          * {
9              margin: 0;
10             padding: 0;
11             box-sizing: border-box;
12         }
13
14         body {
15             font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
16             background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
17             min-height: 100vh;
18             display: flex;
19             justify-content: center;
20             align-items: center;
21             padding: 20px;
22         }
23
24         .container {
25             background: white;
26             border-radius: 10px;
27             box-shadow: 0 10px 40px rgba(0, 0, 0, 0.2);
28             padding: 60px 40px;
29             max-width: 500px;
30             width: 100%;
31             text-align: center;
32         }

```

```
templates > <> success.html > ...
2   <html lang="en">
3   <head>
7     <style>
33
34     .success-icon {
35       width: 80px;
36       height: 80px;
37       background: #4caf50;
38       border-radius: 50%;
39       display: flex;
40       justify-content: center;
41       align-items: center;
42       margin: 0 auto 30px;
43       animation: scaleIn 0.5s ease-out;
44     }
45
46     .success-icon::after {
47       content: '✓';
48       color: white;
49       font-size: 48px;
50       font-weight: bold;
51     }
52
53     @keyframes scaleIn {
54       from {
55         transform: scale(0);
56       }
57       to {
58         transform: scale(1);
59       }
60     }
61
```

```
templates > <> success.html > ...
2   <html lang="en">
3   <head>
7     <style>
53     @keyframes scaleIn {
60   }
61
62   h1 {
63     color: #333;
64     font-size: 32px;
65     margin-bottom: 15px;
66   }
67
68   .message {
69     color: #666;
70     font-size: 16px;
71     margin-bottom: 30px;
72     line-height: 1.6;
73   }
74
75   .details {
76     background: #f5f5f5;
77     padding: 20px;
78     border-radius: 5px;
79     margin-bottom: 30px;
80     text-align: left;
81   }
82
83   .details p {
84     color: #555;
85     font-size: 14px;
86     margin-bottom: 8px;
87   }
```

```
templates > <> success.html > ...
2 <html lang="en">
3 <head>
7 <style>
83 .details p {
87 }
88
89 .details strong {
90   color: #333;
91 }
92
93 .button-group {
94   display: flex;
95   gap: 10px;
96 }
97
98 a,
99 button {
100   flex: 1;
101   padding: 12px;
102   border: none;
103   border-radius: 5px;
104   font-size: 16px;
105   font-weight: 600;
106   cursor: pointer;
107   text-decoration: none;
108   transition: transform 0.2s, box-shadow 0.2s;
109   display: inline-block;
110 }
111
```

```
templates > <> success.html > ...
2 <html lang="en">
3 <head>
7 <style>
98 a,
110 }
111
112 .btn-primary {
113   background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
114   color: white;
115 }
116
117 .btn-primary:hover {
118   transform: translateY(-2px);
119   box-shadow: 0 10px 20px rgba(102, 126, 234, 0.3);
120 }
121
122 .btn-secondary {
123   background: #f0f0f0;
124   color: #333;
125   border: 2px solid #e0e0e0;
126 }
127
128 .btn-secondary:hover {
129   background: #e8e8e8;
130   border-color: #d0d0d0;
131 }
```


templates > <> success.html > ...

```
2  <html lang="en">
3  <head>
7    <style>
128  .btn-secondary:hover {
131  }
132  </style>
133 </head>
134 <body>
135   <div class="container">
136     <div class="success-icon"></div>
137
138     <h1>Data Submitted Successfully!</h1>
139
140     <p class="message">
141       Your information has been received and stored in our database.
142       We'll review your submission and get back to you shortly.
143     </p>
144
145     <div class="details">
146       <p><strong>Status:</strong> ✓ Submission Completed</p>
147       <p><strong>Date & Time:</strong> <span id="timestamp"></span></p>
148       <p><strong>Message:</strong> Thank you for your submission</p>
149     </div>
150
151     <div class="button-group">
152       <a href="{{ url_for('form') }}" class="btn-primary">Submit Another</a>
153       <a href="/" class="btn-secondary">Go Home</a>
154     </div>
155   </div>
156
```

templates > <> success.html > ...

```
2  <html lang="en">
134 <body>
135   <div class="container">
149   </div>
150
151   <div class="button-group">
152     <a href="{{ url_for('form') }}" class="btn-primary">Submit Another</a>
153     <a href="/" class="btn-secondary">Go Home</a>
154   </div>
155 </div>
156
157 <script>
158   // Display current date and time
159   const now = new Date();
160   const options = {
161     year: 'numeric',
162     month: 'long',
163     day: 'numeric',
164     hour: '2-digit',
165     minute: '2-digit',
166     second: '2-digit'
167   };
168   document.getElementById('timestamp').textContent = now.toLocaleDateString('en-US', options);
169 </script>
170 </body>
171 </html>
172
```


127.0.0.1:5000

Submit Your Data

Fill out the form below to submit your information

Full Name *

John Doe

Email Address *

john@example.com

Phone Number

(123) 456-789

Message *

Hii everyone,Nice to meet you

Submit Data

127.0.0.1:5000/success

✓

Data Submitted Successfully!

Your information has been received and stored in our database. We'll review your submission and get back to you shortly.

Status: ✓ Submission Completed

Date & Time: June 5, 2026 at 12:59:05 PM

Message: Thank you for your submission

Submit Another

Go Home